

Enhancing Housing Search Efficiency: Integrating Public Transit Analytics into Real Estate Listings

CS4991 Capstone Report, 2026

Kevin Shi
Computer Science
The University of Virginia
School of Engineering and Applied Science
Charlottesville, Virginia USA
nwk6tq@virginia.edu

ABSTRACT

Apartment hunting in metropolitan areas like New York City is an inefficient process in part due to the lack of personalized location context on major listing platforms, requiring users to manually cross reference street addresses with navigation tools. To enhance the housing search process, I developed Pace, a Chrome extension that automatically integrates public transit analytics directly into StreetEasy, the primary tool used by NYC apartment hunters. The application utilizes JavaScript to inject dynamic content into the DOM and leverages the Geoapify API to calculate precise transit times between apartment addresses and a user defined location like a workplace. The tool displays commute times on both individual listing pages and aggregate search result cards, and it offers a direct link to Google Maps with a more detailed view of the fastest route. These features save users 30-60 seconds per listing, cumulatively saving hours over the duration of an apartment search. Future work will focus on expanding the platform compatibility to include Zillow and Redfin, as well as incorporating driving, biking, and walking options for people with different needs.

1. INTRODUCTION

New York City has one of the most competitive rental markets in the United States, with a large and constantly shifting pool of renters searching for apartments at

any given time. Listings on popular platforms can receive dozens of inquiries within hours of being posted, placing significant time pressure on prospective tenants to evaluate options quickly and efficiently. In a city where a significant number of residents rely on public transit, the daily commute to work is often one of the most important factors in choosing where to live.

Despite the vital role that commute time plays in housing decisions, major listing platforms like StreetEasy do not streamline personalized transit information. Users must instead manually copy a listing address, open a tool such as Google Maps, input listing address as well as workplace destination, interpret the transit directions, then return to the listing page to continue browsing. A single lookup could take 30-60 seconds, and when multiplied across the dozens or even hundreds of listings a typical renter evaluates during a search, the time spent on manual cross-referencing could amount to several hours.

2. RELATED WORKS

Several prior works informed the development of Pace, spanning academic research on housing platform design, industry-level commute tools, and independent browser extension projects targeting StreetEasy specifically.

Lembcke et al. (2021) investigated the effect of integrating a commute time search feature into online housing platforms. By conducting an online experiment comparing a platform with and without commute-based filtering, the authors found that commute time integration contributed to significantly higher scores in both information quality and technology acceptance. They concluded that embedding transit data directly into the housing search workflow reduces cognitive load and improves the user experience. These findings informed the core design goal of Pace: rather than requiring users to leave a listing page to assess commute times, the extension integrates transit data directly into the browsing workflow.

At the industry level, Trulia (n.d.) developed a commute tool and heatmap that gives users a visualization of travel times from a given listing to a user-specified destination, supporting multiple transportation modes including public transit, driving, biking, and walking. According to Trulia, this implementation uses a modified version of Dijkstra's algorithm to calculate shortest paths within transportation network graphs. Their approach demonstrates the technical feasibility and user demand for commute analytics in housing search, and Pace applies a similar concept to StreetEasy where no comparable commute tool currently exists.

Independent developers have also recognized the gap in StreetEasy's transit integration. Whong (2015) created a Chrome extension that injects NYC subway line and station data into StreetEasy's map view, allowing users to visually assess an apartment's proximity to transit stops while browsing listings. More recently, Allotey (2023) launched an extension that displays transit commute times directly on StreetEasy listing pages. Pace builds on the approach demonstrated by these developers by incorporating personalized

routing to a user-defined destination, displaying commute times on both individual listing pages and aggregate search result cards, and providing direct links to Google Maps for detailed route information.

3. PROJECT DESIGN

Pace is a Chrome extension that utilizes JavaScript DOM manipulation and the Geoapify API to automatically calculate and inject public transit commute times directly into StreetEasy listing pages. I designed the system as a distributed application consisting of a client-side Manifest V3 browser extension and a server-side proxy managed via Cloudflare Workers.

3.1 System Architecture & Front-End Implementation

I built the client-side application to handle all user interactions, local data storage, and DOM manipulation. To allow users to customize their search experience, I created a popup interface with HTML, CSS, and JavaScript where they can save their primary workplace destination and configure advanced settings. I used the chrome.storage.sync API to persist these preferences across all devices where the user is logged into Chrome.

To extract listing addresses and dynamically inject transit times, I had to navigate StreetEasy's architecture. I utilized a MutationObserver within a content script to monitor the DOM for changes without relying on full browser reloads. When the script detects a listing page, it extracts the building address from text elements and passes a message to the background service worker to handle geocoding.

Once the commute data is returned, I inject a custom widget directly into the page, as shown in Figure 1. I styled this widget to match StreetEasy's native design language,

displaying the transit time alongside a direct link to Google Maps for detailed instructions. For aggregate search pages that display multiple apartments, I implemented a queue system with a concurrency limit. This ensures the application processes many addresses simultaneously without overwhelming the browser with network requests.

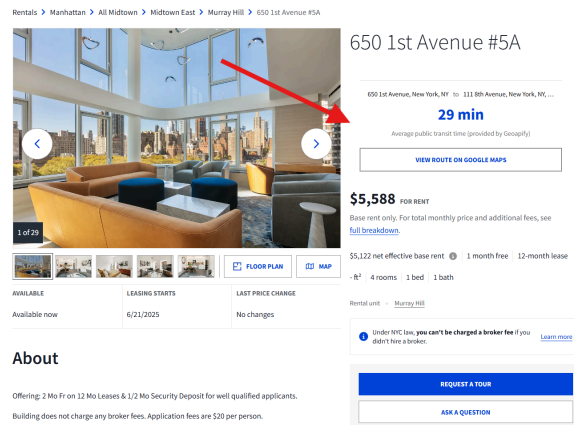


Figure 1: Pace Commute Widget Injected into a StreetEasy Listing.

3.2 Back-End Implementation

To protect API keys and manage quota usage, I developed a server-side intermediary hosted on Cloudflare Workers. This serverless proxy intercepts requests from the extension, appends the necessary API credentials, and forwards them to Geoapify. During the design phase, I considered using AWS Lambda and API Gateway for this backend proxy. However, I selected Cloudflare Workers to avoid the cold start latency of AWS Lambda, which would have delayed autocomplete requests. Cloudflare also provided a more cost-effective free tier compared to AWS API Gateway.

The proxy worker implements a strict allowlist of API endpoints, permitting only specific geocoding and routing operations to prevent bad actors from making unauthorized API calls. If the shared API quota is exceeded, the worker returns a structured

error response, prompting the user to input their own free personal API key.

For spatial calculations, I used the Geoapify API in a two-stage process:

1. Geocoding: I converted the text addresses for the apartment and the user's workplace into latitude and longitude coordinates. To improve accuracy, I applied a "bias" parameter to the request, which prioritizes results within the NYC metropolitan area.
2. Routing: Once I obtained the coordinates, I queried the Public Transit Routing API to calculate the final travel time.

4. RESULTS

I deployed the Pace extension locally via Chrome Developer Mode to evaluate its reliability, accuracy, and efficiency against the traditional manual workflow. Initial testing focused on system stability, which highlighted inconsistencies in the address extraction process. Early versions of the extension relied on standard meta tags or URL parsing, which frequently resulted in incorrect strings that would break the routing logic. I resolved this bug by redirecting the script to target a static DOM element within the StreetEasy "About the building" section, which greatly improved the address capture rate for all subsequent trials. Additionally, baseline accuracy tests revealed minor variances, typically within a few minutes, between the transit times generated by the Geoapify API and the native Google Maps routing engine. Given the project's scope, this slight variance was an acceptable trade-off to maintain a free, open-source infrastructure without relying on a paid commercial API.

Once the system's reliability was established, I conducted a series of solo trials to quantify the efficiency improvements introduced by

the stable build. The primary metrics measured were the active time and the number of user steps required to determine a commute for a single apartment listing. I performed 10 baseline timing trials using the manual method, which involves copying the building address, opening Google Maps, pasting the location, entering a work address, and selecting the transit option. I then compared these metrics against the automated Pace extension workflow. The completed system successfully read the target address and rendered the estimated commute time within 2-4 seconds of the listing page loading. As summarized in Table 1, the extension reduced both user friction and active time spent during the housing search process.

Table 1: Comparison of Manual Commute Search vs Pace Extension.

Metric	Manual Workflow	Pace Extension
User Steps	5 steps	1 step
Active Time Spent	~30 seconds	0 seconds (2-4 second passive wait)
Search Mechanism	Active manual entry across multiple tabs	Automated DOM scraping and API routing

5. CONCLUSION

Pace demonstrates that the gap between apartment hunting and commute awareness can be closed efficiently at the browser level, without requiring platform cooperation or infrastructure-heavy backends. By automatically injecting personalized transit times directly into StreetEasy, the extension eliminates a repetitive, multi-step manual workflow, reducing the active effort to determine commute viability from roughly 30 seconds and five user actions per listing to a

passive 2-4 second wait. The most meaningful technical contribution of this project was developing a resilient content injection pipeline that correctly handles StreetEasy's single-page application architecture to deliver reliable results without degrading page performance. Building Pace increased my practical understanding of distributed systems design, specifically the trade-offs between client-side simplicity and server-side security that led to the Cloudflare Workers proxy architecture, and reinforced that small, well-targeted tools can meaningfully improve everyday workflows without requiring changes to the underlying platforms they augment.

6. FUTURE WORK

The most immediate next steps for Pace involve expanding both platform compatibility and transportation mode support. Extending the extension to Zillow and Redfin would require new address extraction logic and DOM injection points tailored to each platform's structure, but the core geocoding and routing pipeline would remain unchanged, making this a feasible engineering effort. Adding driving, biking and walking transit modes through Geoapify's multimodal routing API would broaden the tool's utility beyond NYC's transit-dependent renters to users in cities or suburbs where car ownership is common. Beyond these near-term additions, longer-term improvements could include a commute score overlay on search result maps, support for multiple saved destinations such as a workplace and a gym, and the ability to filter or sort listings by commute duration natively within the search results page. Publishing the extension to the Chrome Web Store with a streamlined onboarding flow would also make the tool accessible to a general audience rather than requiring manual installation through Chrome's developer mode.

REFERENCES

- Allotey, B. (2023, June 18). *TransitEasy* (Version 1.1.0.0) [Chrome extension]. Chrome Web Store. <https://chromewebstore.google.com/detail/transiteasy/bfdlbpbnchhfhkgjindfmflflhpima>
- Lembcke, T.-B., Willnat, M., Lechte, H., Greve, M., Heinsohn, J., & Brendel, A. B. (2021). Mobility need-adaptive housing platforms: The benefit of a commute time search feature. *Sustainability*, 13(6), Article 3169. <https://doi.org/10.3390/su13063169>
- Trulia. (n.d.). Scaling Trulia's commute API to serve more consumers. *Trulia Blog*. <https://www.trulia.com/blog/tech/commute-api/>
- Whong, C. (2015, June 24). A Chrome extension to inject NYC subway data into streeteasy.com. *Chris Whong*. <https://chriswhong.com/local/a-chrome-extension-to-inject-nyc-subway-data-into-streeteasy-com/>